A3-PS001-TC153

Android Ransomware-like Activity Early Warning in a Controlled Lab

WE ARE ROOT

Nil Darji
nildarji2222@gmail.com

Het Asodariya

hetasodariya5067@gmail.com

Faculty Mentor: Palak Parmar

AIORI Hackathon 3

October 5, 2026

Problem Description

Clear outline of the problem being addressed.

See nowaday’s Signature-based antivirus tools cannot detect zero-day Android ransomware because the malicious binary has not yet been catalogued, and app sandboxing prevents security software from inspecting other apps' memory directly. Ransomware, however, always leaves the same tell: a rapid sequence of file open, write and rename operations across local storage.

- Zero-day binaries evade every signature-matching AV engine.
- Android's app sandbox blocks cross-app memory inspection.
- Ransomware's file-operation fingerprint is detectable even when the binary is unknown.
- Detection claims must be measured: delay, files lost before alert, precision, recall.

Background

Context needed to understand the problem and the proposed optimization.

Real-world incident: a RAT (remote access trojan) attack disabled all security software on a one of my friend's father Android device and stole Rs. 45,000 via unauthorized banking access. The attacker's first move was killing every visible security tool on the device before acting. This is the reason to choose this problem statement to understand how attackers actually attack on real world what lacs in security tools .

This motivates a detector that does not depend on being visible or trusted by the device, and that does not rely on matching a known malicious binary. Instead, it watches the file system directly: Android's inotify subsystem reports file create, write, rename and delete events in real time, which is enough to fingerprint ransomware-style behavior without ever inspecting the attacking app itself.

Literature & RFC Review

Existing standards and prior art most relevant to this problem statement.

- inotify(7) — the Linux kernel's file system event-notification API; the basis for kernel-level, low-overhead file monitoring used here.
- Android Debug Bridge (ADB) root access on emulators/rooted lab devices — documented by the Android Open Source Project as the supported way to get full file-system visibility in a controlled test environment.
- Industry behavioral-ransomware research consistently identifies rapid file rewrite + rename bursts as the most reliable cross-family ransomware indicator, more robust than static signatures against zero-day variants.
- Existing mobile AV products largely rely on signature and permission-based heuristics, which this project's threat model (a RAT that kills visible security apps first) shows is insufficient on its own.

Proposed Solution

Detailed description of the overall solution.

KYN is a behavior-based early-warning detector that runs inside an instrumented, rooted Android emulator. It watches one explicitly user-designated test directory at the kernel level, classifies file-system activity using a sliding-window behavior engine and planted canary files, and responds automatically.

- Kernel-level inotify watcher (C) streams timestamped file events for the test directory.
- Python behavior engine applies dual sliding windows (fast burst / slow evasion) plus a canary-file trap.
- Response engine kills only the offending test workload and restores files from a hash snapshot.
- Scoped by design: no device-wide prevention claims, no monitoring of other apps on stock Android.

Optimization Proposed

Comparing a naive threshold detector against KYN's layered approach.

Before (naive threshold)	After (KYN's layered approach)
Single fixed count of renames/writes in a short window	Dual sliding windows: fast burst (>N in 5s) AND slow cumulative evasion (5 min)
No way to tell ransomware from a legitimate bulk file operation	Planted canary files — any touch is unambiguous, zero false positives
Binary alert only ("something happened")	Pattern classifier labels the attack style for the evidence log
Manual cleanup after an alert	Automatic hash-snapshot restore + scoped kill of only the offending PID

Solution Architecture & Design

Flowchart showing the detection and response pipeline end to end.

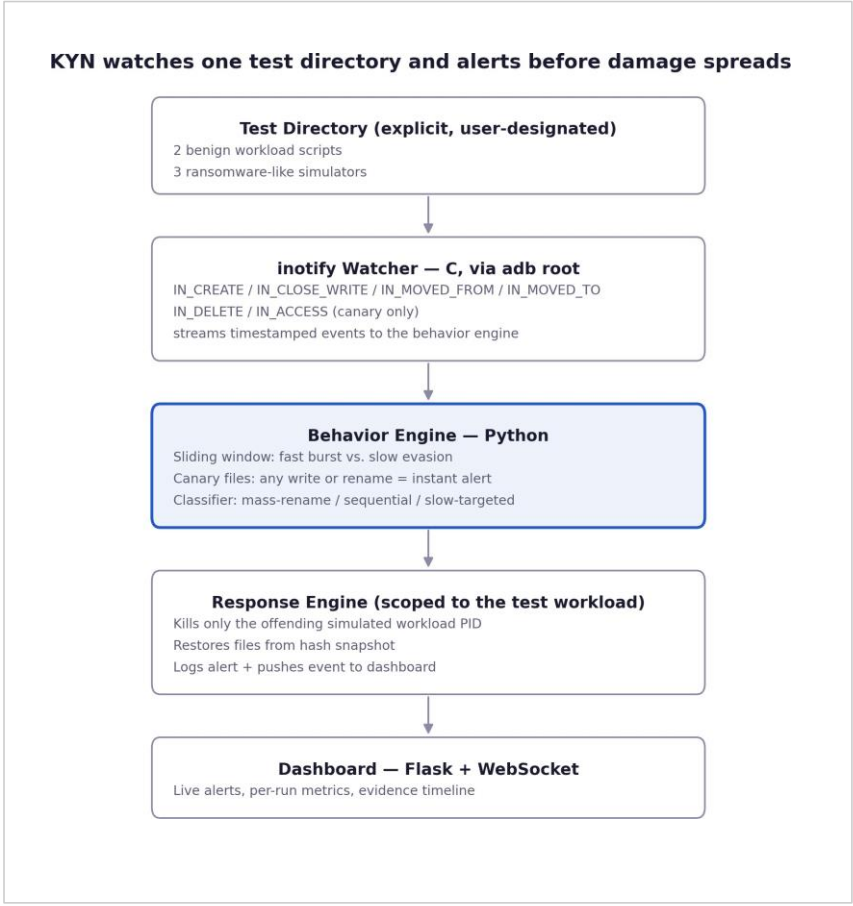


Figure: System Architecture Diagram

Data Flow Diagram

Every file-system event passes through all four checks before a verdict is logged.

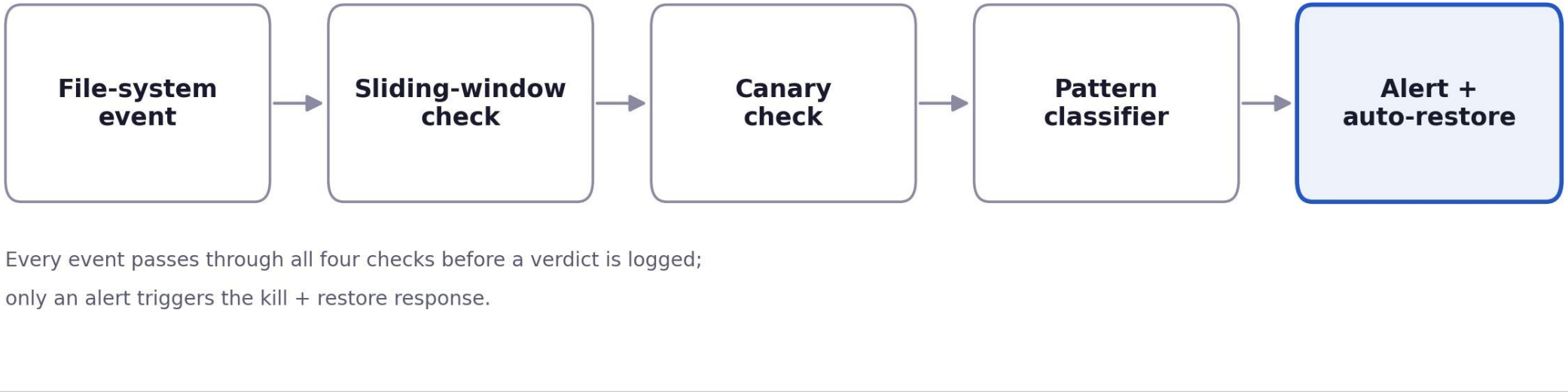


Figure: Data Flow Diagram (DFD)

Deployment Timeline

Build milestones and methodology across the development window.

Period	Focus	Deliverable
Sep 25 – 30	Android Studio + rooted AVD	Emulator running, ADB root working
Oct 1 – 7	inotify C watcher + simulators	Terminal-level detection working
Oct 8 – 14	Behavior engine + canary files	Slow evasion caught
Oct 15 – 21	Metrics + hash-restore engine	Delay / precision / recall measured
Oct 22 – 26	Dashboard (Flask + WebSocket)	Full pipeline, live alerts
Oct 27 – 31	Polish + demo video + docs	Submission ready

Bibliography

Bibliography I

[1] inotify(7) — Linux Programmer's Manual. man7.org.

[2] Android Open Source Project. “Android Debug Bridge (adb).” developer.android.com.

[3] Flask & Flask-SocketIO documentation. flask.palletsprojects.com.

[4] Real-world incident reference used as this project's threat-model baseline: RAT attack disabling Android security apps prior to unauthorized banking access (Rs. 45,000 loss).

Thank You for Your Attention!

WE ARE ROOT · A3-PS001-TC153